

Programação Funcional

1º Ano

2024/2025

Maria João Frade, João Saraiva - Dep. Informática, UM

O que é a programação funcional?

- É um estilo de programação em que o mecanismo básico de computação é a [aplicação de funções](#) a argumentos.
- Uma linguagem de programação que suporte e encoraje esta forma de programação diz-se [funcional](#).
- É um [estilo de programação declarativo](#) (em que um programa é um conjunto de declarações que descrevem a relação entre input e output), ao invés de um estilo imperativo (em que o programa é uma sequência de instruções que vai alterando o estado, o valor das variáveis).

Exemplo da função ??!

$$0! = 1$$
$$n! = n \cdot (n-1)!$$

Haskell (uma linguagem declarativa)

```
fact 0 = 1
fact n = n * fact (n-1)
```

As equações que são usadas na definição de `fact` são **equações matemáticas**. Elas indicam que o lado esquerdo e o lado direito do `=` têm o mesmo valor. O valor dos identificadores é **imutável**.

```
int factorial(int n)
{ int i, r;
  i = 1;
  r = 1;
  while (i<=n) {
    r = r*i;
    i = i+1; }
  return r;
}
```

Isto é muito diferente do uso do `=` nas linguagens imperativas como o C. Neste caso, o valor dos identificadores é **mutável**. Por exemplo, a instrução `i = i+1` representa uma atribuição (o valor anterior de `i` é destruído, e o novo valor passa a ser o anterior mais 1). Portanto, o valor de `i` é alterado.

Programa resumido

Esta UC corresponde a uma introdução ao paradigma funcional de programação, tendo por base a linguagem programação **Haskell** (uma linguagem puramente funcional).

1. **Aspectos básicos da linguagem Haskell:** Valores, expressões e tipos. O mecanismo de avaliação. Inferência de tipos. Definições multi-clausais de funções. Polimorfismo.
2. **Listas.** Funções recursivas sobre listas. Modelação de problemas usando listas.
3. **Algoritmos de ordenação de listas:** *insertion sort*, *quick sort* e *merge sort*.
4. **Ordem superior:** Padrões de computação. Programação com funções de ordem superior.
5. **Tipos algébricos:** Definição de novos tipos e sua utilização na modelação de problemas.
6. **Árvores:** Árvores binárias, árvores de procura, árvores irregulares e algoritmos associados.
7. **Classes:** O mecanismo de classes no tratamento do polimorfismo e da sobrecarga de funções.
8. **IO:** O tratamento puramente funcional do input/output. O monade IO.

Resultados de aprendizagem

- Resolver problemas de programação decompondo-os em problemas mais pequenos.
- Desenvolver e implementar algoritmos recursivos sobre listas e sobre árvores.
- Desenvolver programas tirando partido da utilização das funções de ordem superior.
- Definir tipos algébricos enquadrá-los na hierarquia de classes e programar com esses tipos.
- Escrever programas interativos.

Método de avaliação (anos anteriores)

$$\text{Nota Final} = (\text{Nota do 1º teste}) \times (\text{Nota do 2º teste})$$

- **1º teste:** uma questão selecionada aleatoriamente de um conjunto de questões simples, previamente divulgado. (Notas 0 ou 1)
- **2º teste:** prova escrita sobre toda a matéria. (Notas de 0 a 20)

Datas previstas para as avaliações

1º teste: aulas TPs de 23 a 28 de Outubro

2º teste: 3 de Janeiro

Exame de recurso: 24 de Janeiro

Método de avaliação (**a confirmar!**)

$$\begin{aligned}\text{Nota Final} = & (\text{Nota do 1º teste}) * 0.20 + \\ & (\text{Nota 1ª parte do 2º teste}) * 0.20 \\ & (\text{Nota 2ª parte do 2º teste}) * 0.60\end{aligned}$$

Nota mínima: 50% na (Nota do 1º teste) + (Nota 1ª parte do 2º teste)

- **1º teste:** Escolha múltipla.
- **1ª parte do 2º teste:** Escolha múltipla.
- **2ª parte do 2º teste:** prova escrita sobre toda a matéria.

Datas previstas para as avaliações

1º teste: 19 de Outubro (Sábado)

2º teste: 11 de Dezembro

Exame de recurso: 15 de Janeiro

Bibliografia

- Fundamentos da Computação. Livro II: Programação Funcional.
José Manuel Valença e José Bernardo Barros. Universidade Aberta.
- Programming in Haskell. Graham Hutton. Cambridge University Press, 2016.
- Haskell: the craft of functional programming. Simon Thompson.
Addison-Wesley.
- www.haskell.org/documentation
- Slides das aulas teóricas e fichas práticas: elearning.uminho.pt

Características das linguagens funcionais

- O mecanismo básico de programação é a **definição e aplicação de funções**.
- **Funções são entidades de 1ª classe**, isto é, podem ser usadas como qualquer outro objecto: passadas como parâmetro, devolvidas como resultado, ou mesmo armazenadas em estruturas de dados.
- Grande **flexibilidade**, capacidade de **abstracção** e **modularização** do processamento de dados.
- Os programas são **concisos**, **fáceis de manter** e **rápidos de desenvolver**.

Cronologia

1930's - *Lambda calculus*: uma teoria matemática the funções. (*Alonso Church, Haskell Curry*)

1950's - *Lisp*: a 1ª ling. prog. funcional, sem tipos, impura. (*John McCarthy*)

1960's - *ISWIM*: a 1ª ling. prog. funcional pura. (*Peter Landin*)

1970's - *FP*: ênfase nas funções de ordem superior e no raciocínio sobre programas. (*John Backus*)

ML: a 1ª ling. funcional moderna, com polimorfismo e inferência de tipos. (*Robin Milner*)

1980's - *Miranda*: ling. funcional com *lazy evaluation*, polimorfismo e inferência de tipos. (*David Turner*)

1990's - *Haskell*: ling. funcional pura, *lazy*, com um sistema de tipos extremamente evoluído, criada por um comité de académicos.

2000's - Publicação do **Haskell Report**: a 1ª versão estável da linguagem, actualizada em 2010.

2010's - **Haskell Platform**: distribuição standard do **GHC** (Glasgow Haskell Compiler), que inclui bibliotecas e ferramentas de desenvolvimento.

Haskell

Haskell is a general purpose, purely functional programming language incorporating many recent innovations in programming language design. Haskell provides higher-order functions, non-strict semantics, static polymorphic typing, user-defined algebraic datatypes, pattern-matching, list comprehensions, a module system, a monadic IO system, and a rich set of primitive datatypes, including lists, arrays, arbitrary and fixed precision integers, and floating-point numbers. Haskell is both the culmination and solidification of many years of research on non-strict functional languages.

(The Haskell 2010 Report)

www.haskell.org



Declarative, statically typed code.

```
primes = filterPrime [2..]
where filterPrime (p:xs) =
  p : filterPrime [x | x <- xs, x `mod` p /= 0]
```

 Haskell Platform

Haskell with batteries included

Contém o compilador de Haskell **GHC**,
que vamos usar.

Glasgow Haskell Compiler

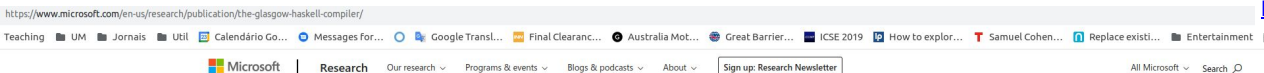
- Principal compilador de Haskell da actualidade.
- Usado na indústria.
- Desenvolvido na Univ. de Glasgow e na Microsoft Research em Cambridge, UK



<https://simon.peytonjones.org/>

Now at [Epic Games](#)
(fortnite?)

[New Functional Language!](#)



The Glasgow Haskell Compiler

Simon Marlow, Simon Peyton Jones
in *The Architecture of Open Source Applications, Volume 2*
Published by Lulu | 2012 | The Architecture of Open Source Applications, Volume 2 edition
[View Publication](#)

[Download BibTex](#)

The Glasgow Haskell Compiler (GHC) started as part of an academic research project funded by the UK government at the beginning of the 1990's, with several goals in mind:

- To make freely available a robust and portable compiler for Haskell that generates high performance code;
- To provide a modular foundation that other researchers can extend and develop;
- To learn how real programs behave, so that we can design and build better compilers.

GHC is now over 20 years old, and has been under continuous active development since its inception. Today, GHC releases are downloaded by hundreds of thousands of people, the online repository of Haskell libraries has over 3,000 packages, GHC is used to teach Haskell in many undergraduate courses, and there are a growing number of instances of Haskell being depended upon commercially.

[View Publication](#)

Research Areas

[Programming languages and software engineering](#)

Glasgow Haskell Compiler



- Inclui um compilador e um interpretador de Haskell:
 - **GHC** - o **compilador** que a partir do programa Haskell cria código executável.
 - **GHCi** - o **interpretador** que actua como uma “máquina de calcular”. Tem uma natureza interactiva adequada ao desenvolvimento passo a passo de um programa. É o que usaremos nas aulas.

O ciclo de funcionamento do interpretador é o seguinte:

***lê** uma expressão, **calcula** o seu valor e **apresenta** o resultado*

O interpretador GHCi

O interpretador arranca, a partir de um terminal, com o comando **ghci**

```
$ ghci
GHCi, version ... : http://www.haskell.org/ghc/. :? for help
ghci>
```

- O *prompt* **>** indica que o GHCi está pronto para avaliar.
- **Prelude** é o nome da biblioteca que é carregada, por omissão, no arranque do GHCi e que disponibiliza uma vasta lista de funções.

```
ghci> 5+3*2
11
ghci> sqrt 9
3.0
```

A biblioteca Prelude

O Prelude é a biblioteca Haskell que contém as declarações de tipos, funções e classes que constituem **o núcleo central da linguagem Haskell**. É sempre carregada por omissão.

Por exemplo, tem muitas funções sobre listas:

```
> length [4,2,6,3,1]
5
> head [4,2,6,3,1]
4
> tail [4,2,6,3,1]
[2,6,3,1]
> reverse [4,2,6,3,1]
[1,3,6,2,4]
> last [1..5]
5
> sum [4,2,6,3,1]
16
```

```
> product [1..5]
120
> [1,2,3] ++ [4,5]
[1,2,3,4,5]
> head (tail [1..5])
2
> take 2 [3,4,7,1,8]
[3,4]
> drop 2 [3,4,7,1,8]
[7,1,8]
> length [4,2,1] + head [7,5]
10
```

Aplicação de funções

A notação usada em Haskell para a aplicação de funções difere da notação matemática tradicional.

- Na [matemática](#), a aplicação de funções é denotada usando parêntesis e multiplicação denotada por um espaço.
- Em [Haskell](#), a aplicação de funções é denotada por um espaço e multiplicação denotada por `*`.

Notação matemática

$f(a,b) + c\ d$

Notação Usual Haskell

`f a b + c*d`

Em Haskell [a aplicações de funções tem prioridade máxima](#) sobre todos os outros operadores.

`f a + b` significa `(f a) + b`

Aplicação de funções

Notação matemática

$f(x)$

$f(x,y)$

$f(g(x))$

$f(x,g(x))$

$f(a) + b$

Notação Haskell

`f x`

`f x y`

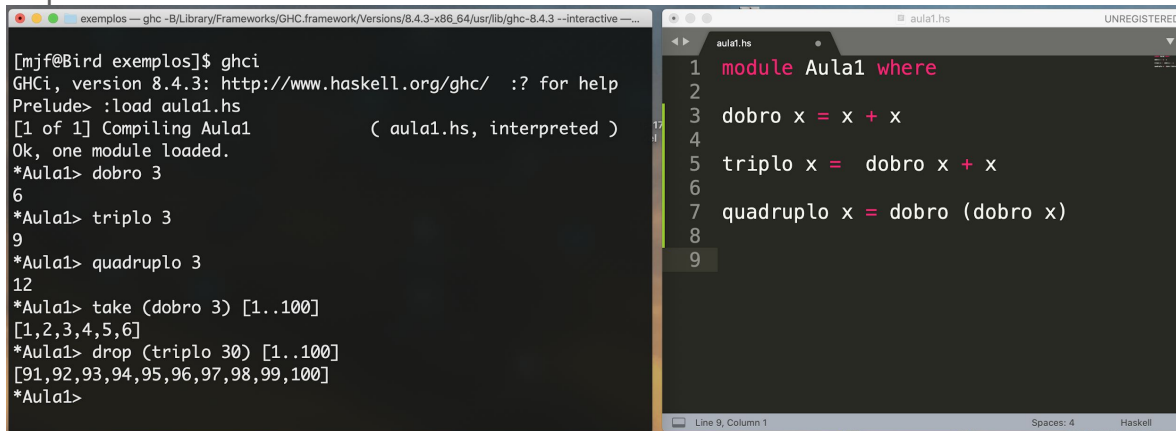
`f (g x)`

`f x (g x)`

`f a + b`

Haskell scripts

- Um **programa Haskell** é constituído por um, ou mais, ficheiros de texto que contêm as definições das novas funções, tipos e classes usados na resolução de um dado problema.
- A esses ficheiros Haskell costumam-se chamar **scripts**, pelo que o nome dos ficheiros Haskell termina normalmente com **.hs** (de Haskell script).
- No desenvolvimento de um programa Haskell é útil manter duas janelas abertas: uma com o **editor de texto** onde se vai desenvolvendo o programa, e outra com o **GHCi** para ir testando as funções que se vão definindo.



The screenshot displays two windows side-by-side. The left window is a terminal running the GHCi interpreter. The right window is a text editor showing a Haskell script named 'aula1.hs'.

```
[mjf@Bird exemplos]$ ghci
GHCi, version 8.4.3: http://www.haskell.org/ghc/  :? for help
Prelude> :load aula1.hs
[1 of 1] Compiling Aula1          ( aula1.hs, interpreted )
Ok, one module loaded.
*Aula1> dobro 3
6
*Aula1> triplo 3
9
*Aula1> quadruplo 3
12
*Aula1> take (dobro 3) [1..100]
[1,2,3,4,5,6]
*Aula1> drop (triplo 30) [1..100]
[91,92,93,94,95,96,97,98,99,100]
*Aula1>
```

```
1 module Aula1 where
2
3 dobro x = x + x
4
5 triplo x = dobro x + x
6
7 quadruplo x = dobro (dobro x)
8
9
```

Line 9, Column 1 | Spaces: 4 | Haskell

Haskell scripts

Mantendo o GHCi aberto podemos acrescentar mais definições ao ficheiro **aula1.hs** e depois recarrega-lo no GHCi para as testar.

Por exemplo, podemos acrescentar ao ficheiro a definição da função factorial

```
fact 0 = 1
fact n = n * fact (n-1)
```

E depois recarrega-lo no GHCi

```
*Aula1> :reload
[1 of 1] Compiling Aula1      (aula1.hs, interpreted)
Ok, one module loaded
*Aula1> fact 5
120
*Aula1>
```

Repare na mudança de nome do prompt para ***Aula1>** que é o nome do módulo que está no ficheiro aula1.hs. Neste momento tem disponíveis no interpretador todas as funções do Prelude e do módulo Aula1.

Alguns comandos do GHCi

Comando

<code>:?</code>	Mostra todos os comandos disponíveis
<code>:load <i>nome</i></code>	Carrega no GHCi o ficheiro <i>nome</i>
<code>:reload</code>	Carrega de novo o ficheiro corrente
<code>:type <i>expressão</i></code>	Indica o tipo de uma expressão
<code>:quit</code>	Sai do GHCi

Pode usar apenas a primeira letra do comando. Por exemplo, `:l aula1.hs`